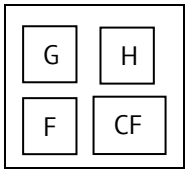
A* Algorithm

(a) Approach:



1 7 8 >	0 6 6	1 5 6 <	2 4 6 <	3 5 8 <
2 6 8 ^				4 4 8 ^
3 5 8 ^	4 4 8 <	5 3 8 <	6 2 8 >	5 3 8 ^
	5 3 8 ^			
7 3 10 >	6 2 8 ^	7 1 8 <	8 0 8 <	

Scores:

- G: distance between the current node and the start node
- H: estimated distance between the current node and the end node. (heuristic)
- F: G + H; total cost of the node

CF – comeFrom

Note:

- If we reach a node for the first time, or reach a node in less time than it currently took, update G to the cost to reach this node.

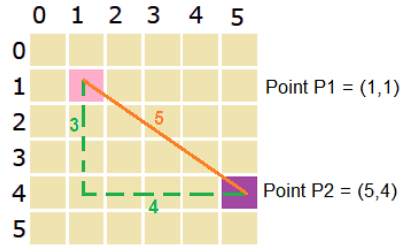
- For heuristic implementation, popular ones include Manhattan distance and Euclidean distance. I believe that the rover will move in 4 directions, i.e. UP, DOWN, LEFT, RIGHT, hence I chose Manhattan distance.

- While traversing, the next visited node is always the node with the lowest F score. If the F scores are the same, choose the node with lower G score.

- Start node is the only node that did not "come from anywhere".

- We only update the F score of a neighbour node if the previous G (of that neighbour node) + 1 < current G (of the current node). If update occurs, change the sign of "comeFrom".

Manhattan Distance vs Euclidean Distance:



Euclidean distance = $\sqrt{(5-1)^2 + (4-1)^2} = 5$

Manhattan distance = $|5-1| + |4-1| = 7$

(b) Complexity:

Time Complexity: O(wh log(wh))

Space Complexity: O(wh),

where w is the width and h is the height

(c) Pseudocode:

1. Create a minHeap to store the nodesToVisit, with the minimum F value node at the top of the heap.
2. Start traversal from the startNode, and store the startNode in the nodesToVisit initially.

3. Loop until the endNode is found.

```
while the heap is not empty:
   visit the nodes with least F, and remove them from the heap
   get the neighbouring nodes
        traverse the neighbouring nodes:
            skip wall and nodes with large G
            calculate the F score for the neighboring nodes using Manhattan Distance (H) + G
            if the nodes have not been visited:
                    insert the nodes into the heap
            else:
                    update the F score of the node
```

4. Construct path through traversing back the nodes and see the sign of the comeFrom, by reversing the list eventually.